

A Technical Report on Lightweight scGPT Fine-Tuning for PBMC68k Cell-Type Classification

Prepared from Remote Experimental Runs

May 5, 2026

Abstract

This report documents a compact scGPT fine-tuning workflow conducted on the public `scanpy` PBMC68k reduced dataset for single-cell cell-type classification. Two successful experiments were completed on a remote Linux host: (1) a baseline run initialized from scratch and (2) a follow-up run that reused the best checkpoint from the baseline under a more classification-focused objective. Both runs used a small transformer configuration (`d_model=64`, `nlayers=2`, `hvg=128`) and were executed on CPU. The baseline achieved a best validation accuracy of 0.3429 and a final test accuracy of 0.3429 after 5 epochs. The follow-up run, although described operationally as a pretrained-style continuation, used the baseline checkpoint rather than an official external scGPT model zoo checkpoint; it preserved the same test accuracy of 0.3429 after 10 epochs while reducing the final test classification loss from 1.9761 to 1.9040. The experiments therefore demonstrate a reproducible scGPT training pipeline and modest optimization gains in loss, but no measurable classification accuracy gain under the tested low-capacity setting.

1 Introduction

Foundation models for single-cell analysis have attracted substantial interest because they offer a unified representation-learning framework across heterogeneous transcriptomic tasks. scGPT is one such model family, designed to model gene-expression profiles with transformer-based sequence processing. In practical deployment, however, many users do not immediately run large-scale official checkpoints. Instead, they first validate that a local fine-tuning stack can execute end-to-end on public datasets, save checkpoints reproducibly, and produce stable metrics under constrained hardware conditions.

The work summarized here belongs to that validation stage. The goal was not to claim a new state of the art, but to establish a functioning fine-tuning pipeline for scGPT on a public benchmark-like dataset and to inspect whether a simple continuation strategy could improve downstream classification behavior. The resulting experiments are best interpreted as engineering verification runs with a small model and low-resource setup.

2 Experimental Setting

2.1 Dataset

All successful experiments used `scanpy.datasets.pbmc68k_reduced()`, a public reduced peripheral blood mononuclear cell dataset distributed with Scanpy. Cell-type labels were taken from `bulk_labels`. The workflow converted the dataset to its raw expression layer, attached categorical label IDs, and treated the task as supervised multi-class cell-type classification.

2.2 Preprocessing

The baseline script applied the following preprocessing decisions:

- highly variable gene (HVG) selection with `subset_hvg=128`,
- `hvg_flavor=cell_ranger`,
- expression binning with `n_bins=51`,
- no total-count normalization,
- no `log1p` transformation,
- tokenization with special tokens `<pad>`, `<cls>`, and `<eoc>`.

The follow-up script preserved the same effective feature scale in the successful run because it loaded the baseline vocabulary, matched 128 of 765 genes against that vocabulary, and used `hvg=128`. Thus, although the second script supported more realistic checkpoint loading behavior, the successful continuation experiment remained tightly tied to the first run’s reduced feature space.

2.3 Model and Training

The baseline run used a compact scGPT transformer:

- `d_model=64`,
- `nhead=4`,
- `d_hid=128`,
- `nlayers=2`,
- `nlayers_cls=2`,
- `dropout = 0.2`.

Training used AdamW with a stepwise learning-rate schedule and a stratified train/validation/test split. The loss combined classification cross-entropy and masked expression reconstruction:

$$\mathcal{L} = \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{mlm}}\mathcal{L}_{\text{mlm}}.$$

In the baseline, $\lambda_{\text{cls}} = 1.0$ and $\lambda_{\text{mlm}} = 1.0$. In the continuation run, the same architecture was reused from the baseline checkpoint, but the optimization emphasis was shifted to classification by using $\lambda_{\text{mlm}} = 0.1$ and a smaller learning rate.

2.4 Hardware Context

Both successful runs executed on CPU. During remote inspection, `torch.cuda.is_available()` returned false, so these results should be understood as CPU-constrained validation experiments rather than GPU-optimized training.

3 Runs Conducted

3.1 Baseline Fine-Tuning from Scratch

The main successful baseline run was stored under:

`/mnt/scgpt_trial/runs/pbmc68k_scgpt_finetune_20260427_195037`

Its saved configuration was:

- epochs: 5
- batch size: 128
- learning rate: 5×10^{-4}
- weight decay: 10^{-2}
- mask ratio: 0.15
- classification loss weight: 1.0
- MLM loss weight: 1.0
- HVG count: 128
- random seed: 42

This run produced checkpoints, metrics, logs, a saved vocabulary, and a preprocessed `.h5ad` artifact.

3.2 Checkpoint-Initialized Continuation Run

The second successful run was stored under:

```
/mnt/scgpt_trial/runs/pbmc68k_scgpt_pretrained_20260505_180059
```

Despite the script’s support for pretrained-style loading, this run did *not* use an official external scGPT model zoo checkpoint. Instead, it loaded:

```
/mnt/scgpt_trial/runs/pbmc68k_scgpt_finetune_20260427_195037/checkpoints/best_model.pt
```

The continuation configuration was:

- epochs: 10
- batch size: 128
- learning rate: 3×10^{-4}
- weight decay: 10^{-2}
- mask ratio: 0.15
- classification loss weight: 1.0
- MLM loss weight: 0.1
- HVG count: 128
- device: CPU
- random seed: 42

The script successfully loaded the prior run’s vocabulary and matched 128 of 765 genes against it.

4 Results

4.1 Run-Level Summary

Table 1 summarizes the two successful experiments.

Table 1: Summary of successful scGPT runs on PBMC68k reduced				
Run	Epochs	Best val acc.	Final test acc.	Final test cls loss
Baseline from scratch	5	0.3429	0.3429	1.9761
Checkpoint continuation	10	0.3429	0.3429	1.9040

4.2 Epoch-Level Behavior

The baseline run improved quickly from a weaker first epoch to a plateau:

- epoch 1: validation accuracy 0.1810, test accuracy 0.2095
- epoch 2: validation accuracy 0.3429, test accuracy 0.3429
- epochs 3–5: validation accuracy remained 0.3429

The continuation run started immediately at the plateau level and maintained it for all 10 epochs:

- epoch 1: validation accuracy 0.3429
- epochs 2–10: validation accuracy remained 0.3429

However, the continuation run steadily reduced the classification loss:

- validation classification loss decreased from 1.9471 at epoch 1 to 1.9034 at epoch 10,
- final test classification loss decreased from 1.9761 in the baseline run to 1.9040.

4.3 Interpretation

The most important empirical observation is that the accuracy ceiling in this setup was reached very early and never exceeded. The second run improved optimization smoothness and loss values, but it did not change the decision boundary enough to yield a better discrete accuracy outcome. This is consistent with three constraints of the experiment:

1. the model was intentionally small,
2. the effective gene vocabulary remained only 128 genes,
3. the “pretrained” continuation did not introduce new external biological knowledge because it reused the same locally trained checkpoint.

5 Discussion

These results are best understood as a pipeline validation rather than a biological performance study. The experiments show that:

- scGPT can be fine-tuned reproducibly on a public PBMC dataset using a lightweight CPU-based workflow,
- run directories can preserve scripts, configurations, metrics, checkpoints, vocabularies, and preprocessed data,

- a continuation strategy with reduced MLM emphasis lowers loss but does not guarantee accuracy gains.

From a modeling perspective, the plateau at 0.3429 likely reflects limited representational capacity and limited input resolution rather than mere under-training. The HVG count of 128 is extremely small relative to realistic single-cell modeling practice. Likewise, a two-layer, 64-dimensional transformer is suitable for debugging and smoke tests, but not for extracting the full value of scGPT-style transfer learning.

The attempted next step in the engineering workflow was to replace the locally recycled checkpoint with the official blood-domain scGPT checkpoint. That transfer did not complete because the jump-host path repeatedly disrupted large-file transport. This limitation is infrastructural rather than methodological, but it matters because it prevented a clean comparison between local continuation and true external pretrained initialization.

6 Limitations

This report has several clear limitations:

- The dataset is a reduced public example, not a full-scale benchmark protocol.
- Successful runs were executed on CPU, which constrained iteration speed and practical hyperparameter search.
- The second run reused a local checkpoint rather than an official model zoo checkpoint, so it does not test genuine cross-domain transfer.
- No multiple-seed aggregate was computed.
- No confusion matrix, macro-F1, or class-wise recall analysis was produced.

7 Conclusion

The remote experiments established a functioning, reproducible scGPT fine-tuning workflow for PBMC68k reduced cell-type classification. The baseline from-scratch run and the checkpoint-initialized continuation run both converged to a final test accuracy of 0.3429. The continuation run improved classification loss from 1.9761 to 1.9040 but did not improve accuracy. The practical conclusion is that the implemented pipeline is valid, but the tested configuration is too small and too locally self-referential to realize the intended benefits of scGPT transfer learning. A meaningful next experiment would require successful use of an official scGPT pretrained checkpoint and a larger feature/model configuration.

Appendix: Exact Successful Run Directories

- Baseline: `/mnt/scgpt_trial/runs/pbmc68k_scgpt_finetune_20260427_195037`
- Continuation: `/mnt/scgpt_trial/runs/pbmc68k_scgpt_pretrained_20260505_180059`